dangerous biological information that can be used for nefarious purposes such as that which could be used for bioweapons. The ability to interpret and audit PLM representations in an unsupervised manner such as with SAEs and transcoders could help mitigate risks and prevent misuse.

Despite the promising results, our work has certain limitations. For protein-level representations, which are a major focus of our work, we interpret mean-pooled protein-level representations, a popular choice in downstream protein-level tasks (4), but it remains to be seen whether other extraction methods (e.g., beginning-of-sequence (BOS) token representations) yield different sets of interpretable features. Investigating how different representation strategies influence interpretability could be an important direction for future research. We acknowledge that while interpretability methods like SAEs and transcoders hold promise for interpreting features that a PLM has learned, as unsupervised methods, we do not believe these methods are built for predicting protein function. This is because unlike supervised approaches to function prediction, or prediction of any other sort, SAEs and transcoders are unsupervised methods which stand at a significant disadvantage to even the simplest of supervised methods (21).

The type of SAEs we use here for both the protein-level and amino acid-level representations are the TopK SAEs (10), which are a very popular choice for interpreting LLMs; however, there are a few other kinds of SAEs such as those using the L1 norm (9) or Gated SAEs (22). It is possible that some other SAE architecture, for example, the recently proposed BatchTopK (23), may be more suitable depending on the kind of representation with which one works. It has also been documented that features extracted from SAEs, more so TopK SAEs, in the natural language setting can differ depending on the random seed used to initialize weights (24). Although we do not assess this effect in protein foundation models, it is possible that similar variations exist in our extracted features. Understanding how consistent the extracted features are across different random initializations would be valuable to ensure optimal feature extraction.

The automated interpretability protocol we use to quantify the interpretability of sparse features or neurons is meant to serve as a proxy for this task. Although this approach is scalable, we expect it to have limitations, including the effects of both the language used in the interpretation and simulation prompts to the LLM, as well as the type and quality of metadata provided as context. Due to API constraints, we interpret only a subset (200 neurons) of the 20,000 sparse features for each SAE and transcoder. A more exhaustive interpretation would likely reveal additional biologically relevant features and refine our understanding of the representations. It would also enable us to determine the number of proteins for which at least one interpretable sparse feature activates. It would be helpful to develop an autointerpretability protocol that is able to feasibly accommodate AA-level sparse representations without the need for mean-pooling across the amino acids. This would enable us to extract much more granular information from the trained AA-level SAE and pinpoint one specific amino acid at a time (such as catalytic amino acids in enzymes). Circumventing this limitation of our method would likely allow us to find very particular amino acid determinants of specific GO features.

Contemporaneous to our work, Simon and Zou (25) and Adams et al. (26) train and interpret sparse autoencoders on PLM representations, with the former's work and ours leveraging LLM-based interpretability. Although these studies focus exclusively on training and interpreting SAEs on amino acid-level representations, we train and interpret SAEs on both protein-level and amino acid-level representations, as well as transcoders. Protein-level representations, which are a major focus of our work, are widely used for downstream protein prediction tasks (4), making such SAEs and transcoders particularly relevant for interpretability in the context of such downstream applications. Brixi et al. (27) have also contemporaneously trained and interpreted sparse autoencoders on a genomic foundation model (Evo2).

Future work could explore how different methods for extracting protein-level representations impact the kind of sparse features learned by SAEs. If representation choices (e.g., mean pooling vs. BOS token extraction) significantly affect what sparse features are extracted, this could inform best practices for protein-level downstream applications, depending on the task at hand. Investigating how the nature of sparse features extracted varies across different types of SAEs would also be helpful for guiding further use of SAEs in PLMs. We anticipate the extension of similar interpretability methods to other biological foundation models beyond ESM2 will aid in establishing SAEs and their variants as a standard tool for interpreting protein language and other representations in biology.

## Materials and Methods

**SAE: Data.** We start with a dataset of 50 million protein sequences from the Uniref50 dataset (28), while excluding sequences present in the UniProt dataset (17) or sequences longer than 1024 residues. We split the dataset into an 80-20 training-validation split. Then, to get the protein-level representation for each sequence, we mean-pool (after removing the BOS and end-of-sequence (EOS) representations) the token representations extracted from layer L of the ESM2 model: "esm2 t12 35M" UR50D. This dataset is used for training the SAE and transcoder on protein-level representations for layer L. On the other hand, the SAE on amino acid-level representations is trained directly (i.e. without mean-pooling) on the individual (non-BOS and non-EOS) token-level representations for layer L.

**SAE: Model.** Sparse autoencoders can be of different kinds depending on the sparsity constraint imposed on the hidden layer. The two most popular kinds of SAEs are Gao et al.'s (10) TopK SAE [used by Adams et al. (26)] and Bricken et al.'s (9) L1 SAE [used by Simon and Zou (25)]. We work with the TopK SAE, though we do not normalize the decoder. Let the protein-level representation or amino acid-level representation from layer $L$ be $x_L$. Due to the ESM model we work with, $x_L \in R^{480}$. Let $W_{\text{enc}}$ be the encoder weight matrix, $b_{\text{enc}}$ be the encoder bias, $b_{\text{dec}}$ be the decoder bias and $W_{\text{dec}}$ be the decoder weight matrix. Let $x_{L,\text{norm}}$ be the layer normalized version of $x_L$ and let $\text{Top}_k$ be the function that only allows the top $k$ entries to pass through as is, zeroing out the rest. Then the encoder of the sparse autoencoder $S_{\text{enc}}$ is defined as follows:

$$y := S_{\text{enc}}(x_L) = \text{ReLU}\left(\text{Top}_k\left(W_{\text{enc}}(x_{L,\text{norm}} - b_{\text{dec}}) + b_{\text{enc}}\right)\right)$$

We set $k$ to be 64 for the protein-level representations and $k$ to be 16 for the AA-level representations. Now $y$ is in the latent space $R^{20,000}$, which will contain the sparse features (i.e. sparse neurons).

The decoder $S_{\text{dec}}$ is defined as follows:

$$z := S_{\text{dec}}(y) = \text{ULN}\left(W_{\text{dec}}(y) + b_{\text{dec}}\right),$$

where *ULN* simply undoes the normalization that we performed with layernorm in the first step, by multiplying by the SD and adding the mean.

**SAE: Training.** The main loss function for the sparse autoencoder is the standard mean square error reconstruction loss

$$\text{Mean Square Error (MSE)}(a,z) = \frac{1}{n}\sum_{i=1}^{n}(a_i - z_i)^2,$$

where $a = x_L$ for the sparse autoencoder. Though this definition of the sparse autoencoder is sufficient in principle to create a sparse representation of the input $x_L$, it is known to encounter optimization issues (10) due to which a vast majority of neurons in the latent space end up dead (inactive on any inputs). So to solve this, there is another reconstruction term called aux reconstruction (10) that is described along with further training details in *SI Appendix*, section S3.

**Transcoder Details.** For the transcoders, the dataset construction, model architecture, and training method stay the same as above, except for two differences: First, we replace $a = x_L$ with $a = x_{L+1} - x_L$, where $x_{L+1}$ is the representation from the L+1th layer. We take the difference due to skip connections in ESM2 because we want the transcoder to learn to predict only the portion that was newly added across that layer, instead of the combined sum including the residual from the previous layer. Second, since the input and desired output are from different layers in a transcoder, the *ULN* function is no longer employed for transcoders, encouraging the model to directly learn the unnormalized desired output.

**Data for Automated Interpretations Using Claude.** First, we detail the procedure for protein-level representations and then later mention a preprocessing modification necessary when working with the amino acid-level representations. We fix a random seed of 42 and use it to pick 50,000 random protein sequences from UniProt that are no longer than 1024 residues. This dataset of 50,000 random proteins is derived from UniProtKB's reviewed portion: UniProt, a highly curated protein database of over 500,000 protein entries that emphasizes minimal redundancy. In UniProt, sequences from the same gene and species are consolidated into a single entry. This approach ensures that each protein is represented uniquely, minimizing redundancy across the dataset.

We extract the protein-level representations from ESM2 as described before and the sparse representations from our trained SAE or transcoder for these sequences. Then we apply min-max normalization (19) to all sparse features (that are not completely inactive) and retain activation levels up to 10 decimal places for precision in the interpretation step.

For the purposes of comparing the interpretability between sparse features and ESM neurons for protein-level representations, it is desirable that the ESM neurons exhibit both active and inactive states, like the sparse features do. So we set a threshold: 0 is a natural choice, so we set 0 as the threshold drawing on Cunningham et al. (11) and Bills et al. (19), and set the activations of ESM neurons below that threshold to 0. Then we similarly apply min-max normalization to the ESM neurons and retain activation levels up to 10 decimal places for precision in the interpretation step.

First, the collection of 20,000 sparse features is preshuffled using a fixed random seed of 42. For each SAE or transcoder that we worked with, we pick 200 consecutive (out of the 20,000 preshuffled) sparse features that are sufficiently active. We consider a sparse feature to be sufficiently active if it has at least twice as many sequences that it is active for, as we will need for the analysis to follow: We do not want to analyze sparse features that are only just barely active. Drawing from Bricken et al. (9), we employ a variety of activation intervals for the automated interpretability protocol. We start by defining blocks containing activation intervals as follows: inactive block contains the trivial activation interval 0 to 0, low-block contains the activation interval 0 to 0.2, mid-block contains the activation intervals 0.2 to 0.4, 0.4 to 0.6, 0.6 to 0.8, and high-block contains the activation interval 0.8 to 1. For each sparse feature we attempt to interpret, we aim to randomly sample the following number of sequences from each interval in their respective blocks: 6 sequences (high-block), 3 sequences (mid-block), 2 sequences (low-block), and 15 sequences (inactive block). We remark, as noted by Bricken et al. (9), that due to sparsity the inactive block is effectively equivalent to a random block–picking completely random sequences across the dataset–since sparse features are highly unlikely to activate on random sequences.

Due to the large number of sparse features (20,000), our SAEs exhibit high levels of sparsity, especially those trained on protein-level representations. So to avoid having to discard too many such sparse features,[¶] we need to implement a generous spillover mechanism: Whenever there are not sufficient sequences for an activation interval, we fill the void by spilling over to as many other intervals (both above and below) as necessary, always preferring intervals closer to the original interval first. This spillover mechanism either eventually accumulates 17 active sequences and 15 inactive sequences for a sparse feature, or terminates unsuccessfully. [In case a sparse feature does not have at least $2 \times 17 = 34$ active sequences and 15 inactive sequences, we skip it (similarly to ref. 9) and pick the next sparse feature for analysis from our preshuffled collection above. But in either case, we always move on to the next sparse feature in our preshuffled collection above.]

We then randomly regard 8 active and 7 inactive sequences from above as interpretation sequences and 9 active and 8 inactive sequences as simulation sequences for our sparse feature. In similar fashion, we pick 200 random ESM neurons (out of the 480), as well as active and inactive sequences associated with each ESM neuron to interpret, and similarly regard 8 active and 7 inactive sequences as interpretation sequences and 9 active and 8 inactive sequences as simulation sequences for our ESM neuron.

**Preprocessing Modification Necessary for AA-Level Representations.** For the amino acid-level representations, we perform the analogous steps as above, but instead of extracting the mean-pooled ESM2 representations, we extract the amino acid-level representations from ESM2 and the sparse representation for each amino acid in the sequence from the corresponding SAE. We note that a lot of the SAE-related interpretability work on LLMs has focused on interpreting short sentence fragments at a time, with fixed context lengths like 64 tokens (10, 11). However, in comparison, our protein sequences have a much larger number of tokens (amino acids) and different proteins contain vastly different numbers of amino acids. So to maintain feasibility as in the protein-level case, we mean-pool the individual AA-level sparse representations into a single representation, and aim to form interpretations for the aggregated (mean) value of a sparse feature (aggregated over the whole sequence).[#] Mean-pooling helps us interpret the mean activation of a sparse feature and ESM neuron respectively over the whole sequence. After mean-pooling, we proceed identically as we do for the protein-level representation.

**Getting Interpretations from Claude.** As mentioned above, we interpret 200 random sparse features (or ESM neurons) for all our SAEs and transcoders. We carry out the following steps for all the individual neurons (sparse or ESM) that we interpret: We give Claude 3.5 Sonnet (claude-3-5-sonnet-20240620) the following information about the interpretation sequences (both active and inactive sequences): sequence ID,[‖] scaled activation level, protein names, gene names, protein families, (GO) biological processes, (GO) cellular components, and (GO) molecular functions. We aim to assess the interpretability of neurons (sparse or ESM) in the context of this metadata, so we ask Claude to come up with a human-readable interpretation of what the neuron appears to be capturing. Several parts of our prompts to Claude (for getting interpretations and using them to simulate activation levels on the simulation sequences) are adapted from the work of Bills et al. (19) and Paulo et al. (20) in the NLP setting. Following Bills et al. (19), we ask Claude to keep its response to no longer than a single sentence–in order to prevent it from coming up with long lists of all the possible individual examples it has seen. To enable precision, the scaled activation levels Claude is provided with are specified up to 10 decimal places. We ask that the interpretation be human-readable (11), to emphasize that the interpretation is meant for humans, especially since we are dealing with biological data. Our prompt stays the same regardless of whether the neuron in question is a sparse feature or ESM neuron. The prompts are included in *SI Appendix*, section S4.

**Simulating Activations with Claude's Interpretation on the Simulation Sequences.** After getting Claude's interpretation for a neuron (sparse or ESM), in a new prompt, we give Claude 3.5 Sonnet (claude-3-5-sonnet-20240620) all the same information (except activation levels) as above for

[¶] Despite the spillover mechanism, several SAE features for protein-level representations need to be discarded (see neurons attempted/neurons analyzed in Table 2).

[#] Pooling (in particular max-pooling) has been used as an aggregation method for SAE activations in the NLP setting (29). Contemporaneously, in the PLM setting, Adams et al. (26) mean-pool sparse representations (albeit for a slightly different use case), while Simon and Zou (25) employ max-pooling.

[‖] Unique and stable entry identifier.

the simulation sequences (both active and inactive sequences), and ask it to return its prediction for the activation levels. The prompts are included in *SI Appendix*, section S4.

**Computing Interpretability Scores.** For each neuron (sparse or ESM), we compute the interpretability score to be the Pearson correlation between the activation values returned by Claude in the simulation step above and the actual (rescaled) activation values for the simulation sequences, following work in the NLP setting (9, 11, 19). Though we provide Claude with highly explicit instructions not to do so, if it returns the incorrect number of prediction values for a neuron, then that neuron is skipped. It is worth remarking that in the simulation step, Claude is never made aware of the fact that a neuron gets skipped if it returns the incorrect number of predictions. So Claude cannot use this to game its way out of neurons it finds hard to provide suitable predictions for.

**Reproducibility.** To help with reproducibility, all the SAE and transcoder training runs, as well as the autointerpretation protocol use a fixed random seed of 42 everywhere. The random seed of 42 used for our autointerpretability results is entirely arbitrary as 42 is a common arbitrary seed choice in machine learning experiments, and we do not expect the conclusions of our automated interpretability to change based on different random seeds.

**Code and Data.** All code used in the project for training and interpreting the sparse autoencoders and transcoders in the various ways is available on Github at: https://github.com/onkarsg10/Rep_SAEs_PLMs (30). Our final SAE and transcoder training implementation draws on implementation ideas, and various components and modules of code from the following open-source implementations (31–39), along with modifications.

The datasets used for training the SAEs and transcoders is (Uniref50 in .fasta.gz format) (28) which can be downloaded from the provided link. The file was downloaded in .fasta.gz format around Nov, 2024. The dataset used for the GO analysis and interpreting our SAEs is (UniProt in TSV format) (17), which can be downloaded by going to the provided link, selecting the columns you want, and clicking download, while selecting TSV as the file format. For the protein-level representations, we used the file downloaded on Nov 12, 2024, and for the AA-level representations we used the file downloaded on Feb 11, 2025. For the GO analysis we also make use of the go.obo file and the goa_human.gaf file (18, 40) that are downloadable at the indicated links, which we downloaded around Nov, 2024.

**Data, Materials, and Software Availability.** Code data have been deposited in GitHub (https://github.com/onkarsg10/Rep_SAEs_PLMs) (30). Previously published data were used for this work (17, 18, 28, 40).

**ACKNOWLEDGMENTS.** We are extremely thankful to Brian Hie, Bowen Jing, Helen Kang, Shuvom Sadhuka, Sunny Guha, Ali Shehper, and Soojung Yang for helpful conversations. We would like to especially thank Jinyeop Song for helpful discussions that led to pooling the Sparse Autoencoder amino acid-level representations. We are grateful to the reviewers for their helpful comments and suggestions. This work and M.B. are partially supported by NIH 1R35GM141861 (to B.B.) and O.G. by NIH U01 CA250554 (to B.B.).

Author affiliations: [a]Department of Mathematics, Massachusetts Institute of Technology, Cambridge, MA 02139; [b]Computer Science and Artificial Intelligence Laboratory, Massachusetts Institute of Technology, Cambridge, MA 02139; [c]Center for Microbiome Informatics and Therapeutics, Massachusetts Institute of Technology, Cambridge, MA 02139; and [d]Department of Biological Engineering, Massachusetts Institute of Technology, Cambridge, MA 02139

1. T. Bepler, B. Berger, Learning the protein language: Evolution, structure, and function. *Cell Syst.* **12**, 654–669 (2021).
2. Z. Lin *et al*., Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* **379**, 1123–1130 (2023).
3. R. Wu *et al*., High-resolution de novo structure prediction from primary sequence. bioRxiv [Preprint] (2022). https://www.biorxiv.org/content/10.1101/2022.07.21.500999v1 (Accessed 22 July 2022).
4. L. C. Vieira, M. L. Handojo, C. O. Wilke, Scaling down for efficiency: Medium-sized transformer models for protein sequence transfer learning. bioRxiv [Preprint] (2024). https://doi.org/10.1101/2024.11.22.624936 (Accessed 31 November 2024).
5. A. Elnaggar *et al*., Toward understanding the language of life through self-supervised learning. *IEEE Trans. Pattern Anal. Mach. Intell.* **44**, 7112–7127 (2022).
6. Z. Zhang *et al*., Protein language models learn evolutionary statistics of interacting sequence motifs. *Proc. Natl. Acad. Sci. U.S.A.* **121**, e2406285121 (2024).
7. J. Vig *et al*., Bertology meets biology: Interpreting attention in protein language models. bioRxiv [Preprint] (2020). https://doi.org/10.48550/arXiv.2006.15222 (Accessed 31 November 2024).
8. N. Elhage *et al*., *Toy Models of Superposition* (Transform Circuits, 2022).
9. T. Bricken *et al*., *Towards Monosemanticity: Decomposing Language Models with Dictionary Learning* (Transform Circuits, 2023).
10. L. Gao *et al*., Scaling and evaluating sparse autoencoders. arXiv [Preprint] (2024). https://doi.org/10.48550/arXiv.2406.04093 (Accessed 31 November 2024).
11. H. Cunningham, A. Ewart, L. Riggs, R. Huben, L. Sharkey, Sparse autoencoders find highly interpretable features in language models. arXiv [Preprint] (2023). https://doi.org/10.48550/arXiv.2309.08600 (Accessed 31 November 2024).
12. C. Kissane, R. Krzyzanowski, J. I. Bloom, A. Conmy, N. Nanda, Interpreting attention layer outputs with sparse autoencoders. arXiv [Preprint] (2024). https://doi.org/10.48550/arXiv.2406.17759 (Accessed 31 November 2024).
13. T. Lieberum *et al*., Gemma scope: Open sparse autoencoders everywhere all at once on Gemma 2. arXiv [Preprint] (2024). https://doi.org/10.48550/arXiv.2408.05147 (Accessed 31 November 2024).
14. J. Dunefsky, P. Chlenski, N. Nanda, Transcoders find interpretable LLM feature circuits. arXiv [Preprint] (2024). https://doi.org/10.48550/arXiv.2406.11944 (Accessed 31 November 2024).
15. Z. He *et al*., Llama scope: Extracting millions of features from Llama-3.1-8b with sparse autoencoders. arXiv [Preprint] (2024). https://doi.org/10.48550/arXiv.2410.20526 (Accessed 31 November 2024).
16. M. Chen, J. Engels, M. Tegmark, Low-rank adapting models for sparse autoencoders. arXiv [Preprint] (2025). https://doi.org/10.48550/arXiv.2501.19406 (Accessed 5 February 2025).
17. The UniProt Consortium, Uniprot: The universal protein knowledgebase in 2021. *Nucleic Acids Res.* **49**, D480–D489 (2020).
18. M. Ashburner *et al*., Gene ontology: Tool for the unification of biology. The gene ontology consortium. *Nat. Genet.* **25**, 25–29 (2000).
19. S. Bills *et al*., Language models can explain neurons in language models. OpenAI Public (2023). https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html. Accessed 10 May 2023.
20. G. Paulo, A. Mallen, C. Juang, N. Belrose, Automatically interpreting millions of features in large language models. arXiv [Preprint] (2024). https://arxiv.org/abs/2410.13928 (Accessed 31 December 2024).
21. L. Smith *et al*., Negative results for SAEs on downstream tasks and deprioritising SAE research (GDM mech interp team progress update 2). AI Alignment Forum (2025). https://www.lesswrong.com/posts/4uXCAJNuPKtKBsi28/negative-results-for-saes-on-downstream-tasks. Accessed 28 March 2025.
22. S. Rajamanoharan *et al*., Improving dictionary learning with gated sparse autoencoders. arXiv [Preprint] (2024). https://doi.org/10.48550/arXiv.2404.16014 (Accessed 31 November 2024).
23. B. Bussmann, P. Leask, N. Nanda, Batchtopk sparse autoencoders. arXiv [Preprint] (2024). https://doi.org/10.48550/arXiv.2412.06410 (Accessed 10 December 2024).
24. G. Paulo, N. Belrose, Sparse autoencoders trained on the same data learn different features. arXiv [Preprint] (2025). https://doi.org/10.48550/arXiv.2501.16615 (Accessed 30 January 2025).
25. E. Simon, J. Zou, Interplm: Discovering interpretable features in protein language models via sparse autoencoders. bioRxiv [Preprint] (2024). https://doi.org/10.1101/2024.11.14.623630 (Accessed 15 January 2025).
26. E. Adams, L. Bai, M. Lee, Y. Yu, M. AlQuraishi, From mechanistic interpretability to mechanistic biology: Training, evaluating, and interpreting sparse autoencoders on protein language models. bioRxiv [Preprint] (2025). https://doi.org/10.1101/2025.02.06.636901 (Accessed 20 March 2025).
27. G. Brixi *et al*., Genome modeling and design across all domains of life with Evo 2. bioRxiv [Preprint] (2025). https://doi.org/10.1101/2025.02.18.638918 (Accessed 20 March 2025).
28. B. E. Suzek, H. Huang, P. McGarvey, R. Mazumder, C. H. Wu, Uniref: Comprehensive and non-redundant Uniprot reference clusters. *Bioinformatics* **23**, 1282–1288 (2007).
29. T. Bricken *et al*., *Using Dictionary Learning Features As Classifiers* (Transform Circuits, 2024).
30. O. Gujral, M. Bafna, Rep_SAEs_PLMs. Github. https://github.com/onkarsg10/Rep_SAEs_PLMs. Deposited 28 June 2025.
31. OpenAI, Sparse autoencoder (2024). Github. https://github.com/openai/sparse_autoencoder/tree/4965b941e9eb590b00b253a2c406db1e1b193942. Deposited 30 June 2024.
32. D. Kerrigan, Saefarer (2024). Github. https://github.com/DanielKerrigan/saefarer/tree/41b5c6789952310f515c461804f12e1dfd1fd32d. Deposited 6 September 2024.
33. EleutherAI, SAE (2024). Github. https://github.com/EleutherAI/sae/tree/main. Deposited 13 November 2024.
34. B. Bussmann, Batchtopk (2024). Github. https://github.com/bartbussmann/BatchTopK/tree/main. Deposited 2 September 2024.
35. T. Cosgrove, Sparse-autoencoder-mistral7b (2024). Github. https://github.com/tylercosgrove/sparse-autoencoder-mistral7b. Deposited 7 August 2024.
36. C. T. Joseph Bloom, D. Chanin, Saelens (2024). Github. https://github.com/jbloomAus/SAELens. Deposited 29 December 2024.
37. N. Nanda, 1l-sparse-autoencoder (2023). Github. https://github.com/neelnanda-io/1L-Sparse-Autoencoder. Deposited 28 October 2023.
38. E. Adams, interprot (2024). Github. https://github.com/etowahadams/interprot/tree/cbe293403e6427b3a7891f8dde15d2f8d7f81d96/interprot. Deposited 5 November 2024.
39. T. Lawson, MLSAE (2024). Github. https://github.com/tim-lawson/mlsae/tree/main/mlsae/model. Deposited 20 December 2024.
40. Gene Ontology Consortium *et al*. The gene ontology knowledgebase in 2023. *Genetics* **224**, iyad031 (2023).